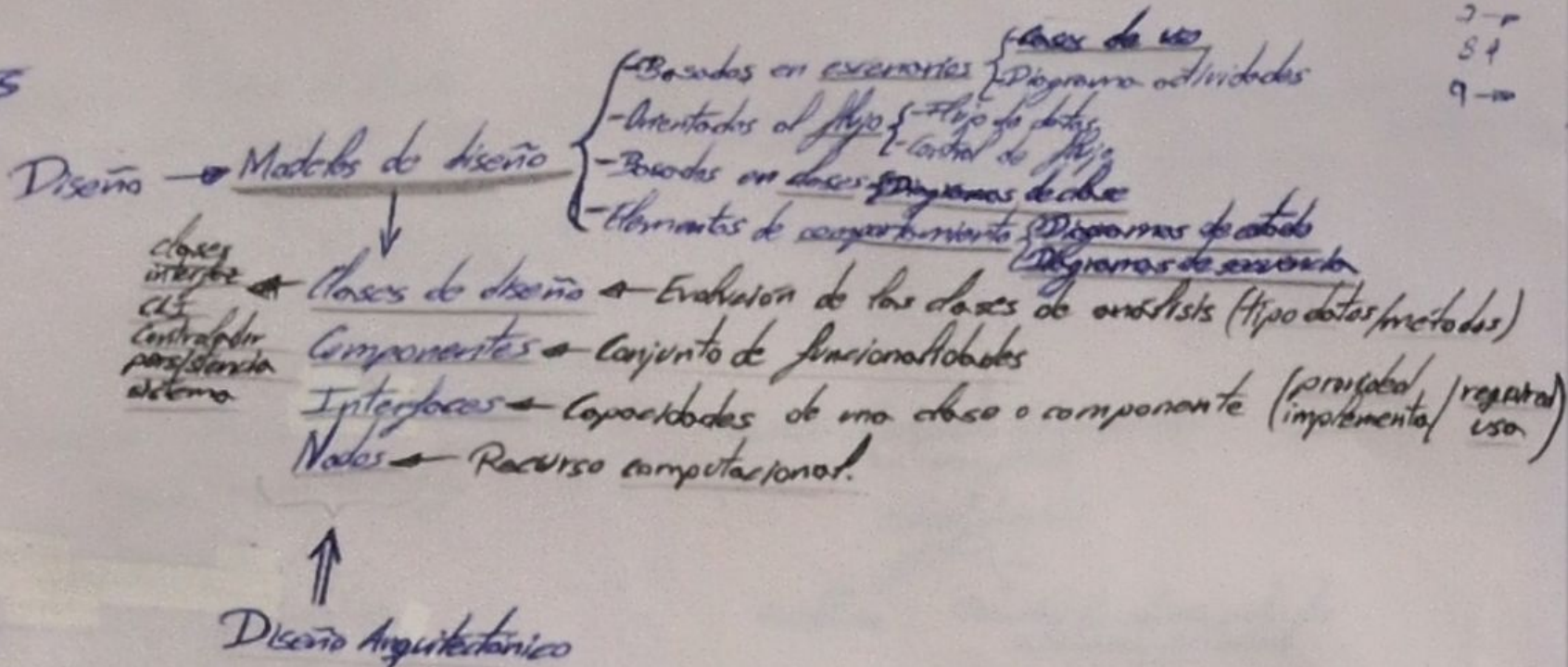


FIS

3-4
5-6
7-8



Trasabilidad

Análisis a Diseño

Diseño a Implementación

- 1° Clases de Dominio o Clases de Diseño
- 2° Incorporación de interfaces
- 3° Incorporación GUI
- 4° Incorporación de persistencia
- 5° Incorporación controladores y clases sistema

- 1° Transformación de relaciones
- 2° Incorporación de persistencia
- 3° Implementación (POO)

Transformación de relaciones

Asociación 1 a 1

Asociación 1 a N (con clase o asociación)

Asociación N a M (con clase o asociación)

Agregación 1 a 1

Agregación 1 a N

Agregación N a M

Composición 1 a N

Herencia (Parcial/total) → Parcial Herencia Instanciable Terminable

Herencia Solapada → Amplias subclases Terminable Subestructura


```

class Persona {
    private DNI dni;
}
class DNI {
    private String numero;
    private Persona persona;
}
Bidireccional
persona.getDNI();
dni.getPersona();

```

```

class Empleado {
    private Departamento dep;
}
class Departamento {
    private List<Empleado> emp
    = new ArrayList<>();
}

```

```

class Alumno {
    private Set<Asignatura> asig
    = new HashSet<>();
}
class Asignatura {
    private Set<Alumno> alum
    = new HashSet<>();
}

```

```

class Profesor {
    private List<Tutoria> tutorias
    = new ArrayList<>();
}

```

```

class Alumno {
    private List<Tutoria> tutorias
    = new ArrayList<>();
}

```

```

class Tutoria {
    private Profesor profesor;
    private Alumno alumno;
    private String observaciones;
}

```

```

class Cache {
    private Motor motor;
}
class Motor {
    private Cache cache;
}

```

El tener ambos solo
gasta a navegabilidad

```

class Pedido {
    private List<LineaPedido> linea
    = new ArrayList<>();
}
class Producto {}
class LineaPedido {
    private Pedido pedido;
    private Producto producto;
    private int cantidad;
    private double precio;
}

```

```

class Biblioteca {
    private List<Libro> libros
    = new ArrayList<>();
}
class Libro {}

```

```

class Serie {
    private Set<Libro> libros
    = new HashSet<>();
}
class Libro {
    private Set<Serie> series
    = new HashSet<>();
}

```

```

class Casa {
    private List<Habitacion> hab
    = new ArrayList<>();
    public void eliminar() {
        hab.eliminar();
    }
}
class Habitacion {}

```


Principios básicos de diseño

Abstracción

Modularidad

Difusión de información

Adecuamiento

Cohesión

$\left\{ \begin{array}{l} \text{- Control} \\ \text{- Datos} \\ \text{- Lógica} \\ \text{- Temporal} \\ \text{- Procedural} \end{array} \right\}$
 $\left\{ \begin{array}{l} \text{- Funcional} \\ \text{- Comunicacional} \\ \text{- Secuencial} \\ \text{- Funcional} \end{array} \right\}$

Principios SOLID

Single Responsibility

Open/Closed

Liskov substitution

Interface Segregation

Dependency inversion

Herencia

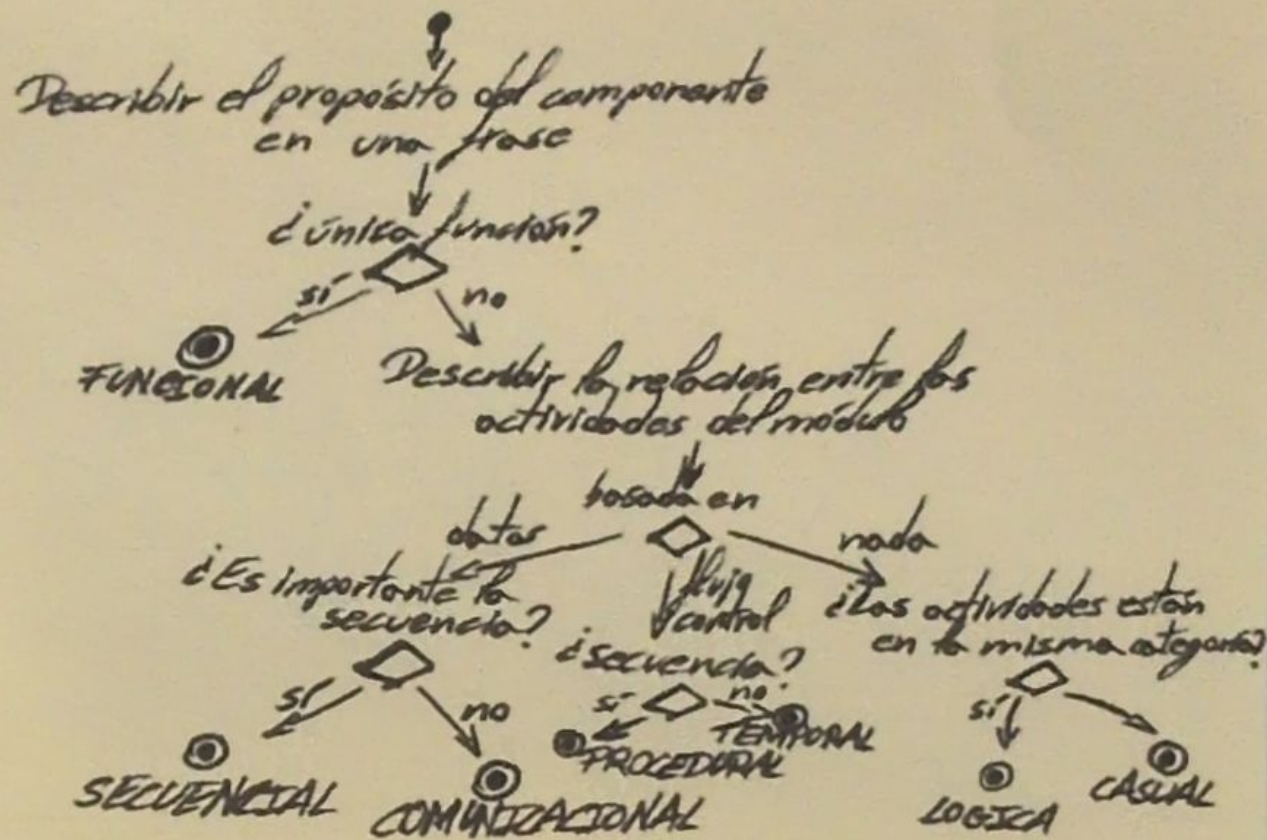
Generalización/Especialización

Composición/Agregación

Agregación/Descomposición, Agrupación/

Individuación

Clasificación/Instantiación



Patrones de diseño software

Patrones creacionales ← Creación de objetos { Singleton ← única instancia
Factory Method ← delegar creación

Patrones estructurales ← Composición y mantenimiento { Proxy ← control de acceso
Adapter ← puente entre interfaces

Patrones de comportamiento ← Comunicación y asignación { Composición
Herencia

Antipatrones de diseño software

Antipatrones de Desarrollo

Antipatrones de Arquitectura

Antipatrones de Gestión

Command ← Acciones como objetos
Observer ← Notificación Automática

Arquitectura de software



Vista ← Aspecto parcial (perspectiva concreta)

Patrones { 3 capas ← Display Logic State
N capas
MVC ← Vista Controlador Modelo
Sense Compute Control

{ Vista Lógica ← Diagrama de ses
Vista de Proceso ← Diagrama secuencia, estados y actividad.
Vista de desarrollo ← Diagrama componentes
Vista física ← Diagrama despliegue
Vista de escenarios ← Diagrama casos de uso.

Estilos ← Forma estándar de organizar un sistema

Componentes

Conectores

Restricciones

Modelos semánticos

Atributos de calidad

- Main Program & Subroutines
- Object-Oriented
- Virtual Machines
- Client-Server
- Pipe and Filter
- Blackboard
- Publish-Subscribe
- Event-Based
- Peer-to-peer

Validación → software corresponde a los requisitos
Verificación → software implementa función correctamente

Revisión Técnica Formal (RTF) → Antes de programar, estático

Pruebas → Después de programar, dinámica

- Pruebas unitarias → Módulos
- Pruebas de integración → nivel de componente
- Pruebas de sistema
- Pruebas de validación

Casos de prueba → Entrada conocida - Salida esperada → Salida obtenida

Identificador

Autor

Versión

Nombre

Propósito

Entorno de Prueba

Pruebas unitarias

- Caja blanca → Se conoce internamente
- Caja negra → solo se conoce la función específica
- Particiones de equivalencia → conjunto de estados válidos o no válidos

Prueba de la ruta base
Prueba de las condiciones
Prueba de los límites
grupos de flujo
condiciones análogas
rutas independientes
casos de prueba

Casos de prueba

- Cobertura de Clases Válidas → caso que cubran el mayor número de clases válidos simultáneamente.
- Cobertura de Clases No Válidas → caso que cubra solamente una entrada no válida.

Pruebas de integración

Driver → llama al componente a probar (bottom-up).

Driver ↑ respuesta del componente

Stub → es llamado por el componente a probar (top-down).

Stub ↓ llamada del componente

- Pruebas de regresión → correr pruebas que antes pasaban al cambiar un componente.
- Pruebas de humo → verifica que lo esencial funcione.

Pruebas del sistema

- Prueba de Recuperación
- Prueba de Seguridad
- Prueba de Rendimiento
- Prueba de Tensión

Pruebas de validación

- Pruebas de Aceptación Contractual
- Pruebas de Aceptación del Usuario
- Pruebas de Aceptación Reglamentaria
- Pruebas de Aceptación Operativa